

CerebroNet — Project Report

Course: DA5402 — AI Application Development
Project: Brain Tumor Classification MLOps Platform
Student: Aryan Prasad
Roll Number: DA25M007

1. Abstract

CerebroNet is an end-to-end MLOps platform that classifies brain MRI scans into four categories — Glioma, Meningioma, Pituitary Tumor, and No Tumor — using deep learning. The system combines a MobileNetV2 classifier (Macro F1: 0.948) with a production-grade infrastructure stack: Docker Compose (7 containers), DVC (7-stage reproducible pipeline), MLflow (experiment tracking), Apache Airflow (ETL orchestration), Prometheus + Grafana (real-time monitoring), and a Streamlit frontend with Grad-CAM explainability. All inference runs in-memory with automatic EXIF stripping to ensure patient data privacy.

2. Problem Statement

Brain tumors account for approximately 1.4% of all cancers globally. Early and accurate classification of tumor type from MRI scans is critical for treatment planning. Manual radiological assessment is time-consuming, subjective, and prone to inter-observer variability. An automated classification system can serve as a screening aid, providing instant second opinions with quantified confidence scores and visual explanations of model reasoning.

Objective: Build a deployable, monitored, and reproducible ML system that classifies brain MRI scans with >90% Macro F1 while adhering to MLOps best practices (versioning, tracking, orchestration, observability, containerization).

3. Dataset

Property	Value
Source	Brain Tumor MRI Dataset (Kaggle)
Raw Images	5,712
Augmented Images	11,200 (after transform stage)
Classes	Glioma, Meningioma, No Tumor, Pituitary
Image Size	224 × 224 (resized)
Split	Training / Testing (with augmentation on training only)
Format	JPEG

Data Engineering Pipeline:

1. **Prepare** (`prepare.py`): Copy raw data into organized class subfolders
2. **Transform** (`transform.py`): Resize to 224^2 , apply augmentations (random flip, brightness $\pm 20\%$, contrast $\pm 20\%$, Gaussian blur), double the training set

4. Methodology

4.1 Model Architectures Trained

#	Model	Architecture	Training Strategy
1	BaseCNN	3-layer CNN from scratch	Baseline reference
2	MobileNetV2	Transfer learning, fine-tune last 3 blocks	Production model
3	EfficientNet-B0	Transfer learning, fine-tune last 3 blocks	Accuracy ceiling
4	SVM	Flattened pixel features + RBF kernel	Classical ML baseline
5	MLP	3-layer fully connected network	Shallow NN baseline

4.2 Training Configuration (from `params.yaml`)

Parameter	Value
Batch Size	16
Epochs	20
Learning Rate	0.0001
Early Stopping Patience	5
Optimizer	Adam
Image Size	224×224
Normalization	ImageNet mean/std
Random Seed	42

4.3 Production Model Selection

MobileNetV2 was chosen over EfficientNet-B0 despite a 1.2% lower F1 because:

- **8× faster training** (80.5s vs 650s per epoch)
- **2.5× faster inference** (<200ms vs ~500ms)
- For a real-time clinical screening tool, sub-200ms latency is critical
- The marginal accuracy gain does not justify the compute overhead

4.4 Explainability — Grad-CAM

Gradient-weighted Class Activation Mapping (Grad-CAM) is implemented on the last convolutional layer (`model.features[-1]`) of MobileNetV2. This generates a heatmap overlay showing which spatial regions of

the MRI scan most influenced the model's prediction, enabling clinicians to verify that the model is focusing on diagnostically relevant areas rather than artifacts.

5. Results

5.1 Model Comparison

Model	Macro F1	Val Accuracy	Speed/Epoch	Inference Latency	Deployed
EfficientNet-B0	0.960	96.5%	650s	~500ms	✗
MobileNetV2	0.948	95.2%	80.5s	<200ms	✓
SVM	0.859	87.1%	—	<50ms	✗
MLP	0.840	85.3%	45s	<30ms	✗
BaseCNN	0.810	82.4%	120s	<100ms	✗

5.2 MLflow Experiment Tracking

All 5 training runs are logged to MLflow with:

- **Parameters:** learning rate, batch size, epochs, optimizer, model architecture
- **Metrics:** macro_f1, val_acc, epoch_time, total_train_time
- **Artifacts:** model checkpoints (.pth/.pkl), confusion matrices, classification reports

5.3 Test Results

19 automated test cases — **all passed:**

- 12 API tests (health, predict, security validation, batch processing)
- 4 UI color-mapping tests
- 3 integration tests (privacy, telemetry, pipeline)

6. System Architecture

6.1 Container Topology (7 Services)

Container	Service	Port	Purpose
cerebronet_frontend	Streamlit	8501	Web UI (8 pages)
cerebronet_backend	FastAPI + PyTorch	8000	REST inference API (7 endpoints)
cerebronet_prometheus	Prometheus	9090	Metrics scraping (15s interval)
cerebronet_grafana	Grafana	3001	12-panel observability dashboard
cerebronet_node_exporter	Node Exporter	9100	Host CPU/memory metrics
cerebronet_mlflow	MLflow	5000	Experiment tracking + model registry

Container	Service	Port	Purpose
cerebronet_airflow	Apache Airflow	8080	ETL pipeline orchestration (2 DAGs)

6.2 API Endpoints

Endpoint	Method	Description
/health	GET	Liveness probe
/ready	GET	Readiness probe
/predict	POST	Single image classification
/predict_cam	POST	Classification + Grad-CAM overlay
/predict_bulk	POST	ZIP batch processing
/metrics	GET	Prometheus scrape target (8 metrics)
/info	GET	Model metadata

6.3 Loose Coupling

Frontend and backend are strictly decoupled:

- Communication exclusively via REST API (<http://backend:8000>)
- No shared filesystem, memory, or direct imports
- [API_URL](#) is configurable via environment variable
- Frontend can be replaced with any HTTP client

7. MLOps Components

7.1 Data Versioning — DVC

[dvc.yaml](#) defines a 7-stage reproducible pipeline:

```
prepare → transform → train_baseline → train_mobilenet → train_efficientnet →  
train_svm → train_mlp → evaluate
```

- All hyperparameters centralized in [params.yaml](#)
- Large data artifacts tracked without Git bloat
- Full pipeline reproducible via [dvc repro](#)

7.2 Experiment Tracking — MLflow

- 5 experiment runs tracked with parameters, metrics, and artifacts
- Model registry for versioned checkpoint management
- [MLproject](#) file defines reproducible entry points
- SQLite backend store at [mlflow.db](#)

7.3 Pipeline Orchestration — Apache Airflow

2 DAGs automate the ML workflow:

- `cerebronet_dag`: Data preparation and model training orchestration
- `cerebronet_scraper_dag`: External data ingestion and ETL

Features:

- Automated email alerts (Success/Failure) configured via SMTP `.env` integration
- Airflow UI accessible at `localhost:8080`

7.4 Monitoring — Prometheus + Grafana

8 custom Prometheus metrics instrumented natively in FastAPI:

Metric	Type	Purpose
<code>cerebronet_requests_total</code>	Counter	Request volume by endpoint/status
<code>cerebronet_inference_latency_seconds</code>	Histogram	P50/P95/P99 latency buckets
<code>cerebronet_predictions_total</code>	Counter	Per-class prediction distribution
<code>cerebronet_confidence_score</code>	Histogram	Confidence score distribution
<code>cerebronet_model_loaded</code>	Gauge	Model availability status
<code>cerebronet_active_requests</code>	Gauge	Concurrent request tracking
<code>cerebronet_payload_size_bytes</code>	Summary	Upload size monitoring
<code>cerebronet_rejected_uploads_total</code>	Counter	Security rejection tracking

Grafana dashboard auto-provisioned on startup with 12 panels covering model status, latency percentiles, throughput, prediction distribution, confidence histogram, and host resource utilization.

7.5 Containerization — Docker Compose

- 7 isolated microservices on `cerebronet_network` bridge
- Docker DNS resolution (frontend → `http://backend:8000`)
- Health checks on backend container
- Volume mounts for model artifacts, configs, and DAGs
- No external cloud dependencies — fully local

8. Frontend UI/UX Design

8.1 Design Philosophy

- **Premium Dark Theme** with purple accent (`#b432ff`) and glassmorphism
- Responsive layout with smooth animations and hover effects
- Centralized CSS (`style.css`, 1275+ lines) for maintainability
- Dynamic brain visualization that shifts color based on prediction result

8.2 Navigation (8 Pages)

Page	Purpose
Predict	Single scan classification with Grad-CAM + Batch ZIP upload for bulk processing
Model Tracker	MLflow experiment comparison with charts
Pipeline	DVC DAG, live Airflow DAG status (real-time REST API), run history, speed metrics
Apps Hub	Direct links to MLflow, Grafana, Airflow, Prometheus, FastAPI Docs
Privacy Policy	Data handling and EXIF stripping documentation
FAQ	Common questions about the platform
Contact Us	Support inquiry form
About	Project credits

8.3 Prediction Modes

Single Scan Mode:

- Upload a brain MRI scan (JPEG/PNG, max 10MB) via drag-and-drop
- Instant classification result with confidence score and inference latency
- Grad-CAM explainability heatmap showing which regions the model focused on
- Probability distribution bar chart across all 4 classes
- Dynamic brain visualization shifts color to match the prediction (Red = Glioma, Yellow = Meningioma, Green = No Tumor, Blue = Pituitary)

Batch Mode (ZIP Upload):

- Upload a ZIP archive containing multiple MRI scans
- Calls the `/predict_bulk` endpoint to process all images in one request
- Displays a results table with filename, prediction, confidence, and latency per image
- Handles mixed valid/invalid files gracefully — invalid images are reported with error messages

8.4 Foolproof Design

- File type validation (JPEG/PNG only)
- 10MB upload size limit with clear error messages
- Graceful handling of API unavailability
- Color-coded prediction results (Red/Yellow/Green/Blue)
- User manual accessible within the platform

9. Security & Privacy

Measure	Implementation
File Extension Whitelist	<code>.jpg</code> , <code>.jpeg</code> , <code>.png</code> only

Measure	Implementation
MIME Type Validation	<code>image/jpeg</code> , <code>image/png</code>
Upload Size Limit	10MB maximum
EXIF Stripping	GPS, timestamps, device info removed
In-Memory Processing	No image written to disk — <code>del</code> called post-inference
CORS Middleware	Configured for cross-origin requests
Rejection Metrics	<code>cerebronet_rejected_uploads_total</code> counter

10. Software Engineering Practices

Practice	Implementation
Version Control	Git + DVC for code and data
Containerization	Docker Compose with 7 services
Logging	Python <code>logging</code> module with structured format
Error Handling	FastAPI <code>HTTPException</code> with descriptive messages
Testing	19 automated test cases (pytest)
Documentation	6 Markdown documents + inline docstrings
Configuration Management	<code>params.yaml</code> for hyperparameters, env vars for runtime
Reproducibility	<code>dvc repro</code> , <code>MLproject</code> , fixed random seed (42)
Code Organization	Modular structure: <code>src/api</code> , <code>src/pipeline</code> , <code>src/training</code>
CI-Ready	All tests runnable via <code>docker exec cerebronet_backend pytest tests/ -v</code>

11. Conclusion

CerebroNet successfully demonstrates a complete MLOps lifecycle:

1. **Data Engineering:** Automated ingestion, versioning, and augmentation pipeline
2. **Model Development:** 5 architectures trained, tracked, and compared
3. **Deployment:** Production model served via REST API with health checks
4. **Monitoring:** Real-time observability with 8 custom metrics and 12-panel dashboard
5. **Explainability:** Grad-CAM heatmaps for clinical interpretability
6. **Privacy:** Zero-persistence architecture with EXIF stripping
7. **Alerting:** Automated email notifications on pipeline success/failure via Gmail SMTP

The platform achieves a 94.8% Macro F1 score with sub-200ms inference latency, balancing accuracy and speed for real-time clinical screening.

12. Challenges Faced & Mitigations

Challenge	Impact	Mitigation
Streamlit HTML sanitization	<code>st.markdown()</code> strips whitespace and styles from <code><pre></code> blocks, breaking the DVC DAG ASCII art	Switched to <code>st.html()</code> which bypasses the Markdown parser entirely and renders raw HTML
EfficientNet inference latency	EfficientNet-B0 achieved highest F1 (0.960) but ~500ms inference was too slow for real-time use	Selected MobileNetV2 (F1: 0.948, <200ms) — 1.2% accuracy gap justified by 2.5× speed gain
Docker container networking	Frontend could not resolve backend hostname on first startup	Used Docker DNS with named network <code>cerebronet_network</code> and <code>depends_on</code> ordering in compose
EXIF metadata privacy risk	Uploaded MRI scans could contain GPS coordinates and patient device info	Implemented <code>strip_exif()</code> that creates a clean image copy with all metadata removed before inference
CSS class inheritance in Streamlit	CSS classes applied via <code>class=</code> attribute were not inheriting <code>text-align</code> from parent divs	Applied critical styles (centering, colors) as inline <code>style=</code> attributes on <code><div></code> elements instead of <code><p></code> tags
Large model artifacts in Git	<code>.pth</code> files (10-25 MB each) bloated the repository	Used DVC to track large artifacts separately; <code>.gitignore</code> excludes raw data directories
Airflow API authentication	Frontend could not pull live DAG status (401 Unauthorized)	Enabled <code>basic_auth</code> backend via <code>AIRFLOW__API__AUTH_BACKENDS</code> environment variable
SMTP email alerting	Gmail blocks normal password authentication for third-party apps	Used Google App Passwords with credentials externalized in <code>.env</code> for security
Resource Starvation (OOM)	MLflow spawned 170 threads consuming 1.5GB RAM, causing 27s latency on 8GB machines	Limited MLflow to <code>--workers 1</code> and created custom <code>Dockerfile.airflow</code> to pre-install dependencies, dropping total RAM usage by >80%

13. Future Work

- GPU-accelerated inference for higher throughput
- DICOM format support for clinical imaging standards
- Federated learning for multi-hospital collaboration without data sharing
- A/B testing framework for model version comparison in production
- Automated retraining pipeline triggered by data drift detection

- Role-based access control for multi-user clinical environments